

RI.UVT.RO Redesign

Week 4 Implementation Plan

Frontend Foundation + Backend Structure

Direcția de Relații Internaționale

West University of Timișoara

Focus: Updated Week 4 plan reflecting Week 3 actual state, carry-over gaps, Vite/Next.js decision risk, role-specific implementation work, and Week 5 handoff conditions.

Active Roles: Role 1, Role 2, Role 4, Role 5, Role 6

Technology Stack Reality: WordPress CMS + REST API + Vite/React Router
+ TypeScript + Tailwind CSS
Team Q-UW9

Contents

1	Week 4 Implementation Plan — Frontend Foundation and Backend Structure	3
1.1	Checkbox Legend	3
1.2	1. Week 4 Objective	3
1.3	2. Week 3 Carry-Over Summary	4
1.4	3. Combined Week 4 Output Checklist	5
1.5	4. Implementation Sequencing Rule	5
1.6	5. Vite vs Next.js — Open Decision	5
1.7	6. Role 1 — Project Manager + Frontend Lead	6
1.7.1	Phase 3 Responsibility	6
1.7.2	Week 3 Carry-Overs	6
1.7.3	System Coordination Tasks	6
1.7.4	Implementation Standards Tasks	6
1.7.5	Sequencing Tasks	6
1.7.6	Deliverables	6
1.8	7. Role 2 — UX Researcher + Frontend Contributor	7
1.8.1	Phase 3 Responsibility	7
1.8.2	Week 3 Carry-Overs — Validate Existing Components	7
1.8.3	Week 4 UX Validation Tasks	7
1.8.4	Component Review Tasks	8
1.8.5	Deliverables	8
1.9	8. Role 4 — UI Designer + Frontend Implementer	8
1.9.1	Phase 3 Responsibility	8
1.9.2	Week 3 Carry-Overs — Documentation First	8
1.9.3	Design Token Tasks	9
1.9.4	UI Atom Tasks	9
1.9.5	Responsive Visual Behavior Tasks	9
1.9.6	Accessibility-Friendly Visual Pattern Tasks	9
1.9.7	Deliverables	9
1.10	9. Role 5 — React Developer	10
1.10.1	Phase 3 Responsibility	10

1.10.2	Week 3 Carry-Overs — Documentation First	10
1.10.3	Frontend Scaffold Tasks	10
1.10.4	Atomic Component Implementation Tasks	10
1.10.5	Reusable Organism Tasks	10
1.10.6	Page Template Tasks	11
1.10.7	Route Implementation Tasks	11
1.10.8	API Layer Tasks	11
1.10.9	Architecture Separation Rules	11
1.10.10	Deliverables	11
1.11	10. Role 6 — WordPress Developer	12
1.11.1	Phase 3 Responsibility	12
1.11.2	Week 3 Carry-Overs — Documentation First	12
1.11.3	CMS Structure Tasks	12
1.11.4	Initial CPT Tasks	12
1.11.5	Taxonomy Tasks	13
1.11.6	Structured Field Tasks	13
1.11.7	REST API Compatibility Tasks	13
1.11.8	Deliverables	13
1.12	11. Suggested File Outputs	13
1.13	12. Definition of Done	14
1.14	13. Week 5 Handoff Notes	15

Document standard: This file follows the RI.UVT.RO document styling system: institutional UVT blue hierarchy, yellow accent rules, readable spacing, structured role sections, and technical code-safe formatting.

1. Week 4 Implementation Plan — Frontend Foundation and Backend Structure

Project: RI.UVT.RO Redesign

Phase: Week 4 / Phase 3

Focus: Frontend Foundation + Backend Structure

Institutional Context: West University of Timișoara — International Relations Department

Active Teams: Team A, Team B, Team C

Active Roles: Role 1, Role 2, Role 4, Role 5, Role 6

Last updated: 2026-05-12 — revised to reflect Week 3 actual state (see [changes/w3/16_Week3_Gap_Report](#))

1.1 Checkbox Legend

Mark	Meaning
[x]	Done — merged into main as of Week 3 end
[~]	Partial — exists in code but missing documentation or incomplete
[]	Not done
[W3]	Week 3 carry-over — was required last week and was not delivered

1.2 1. Week 4 Objective

Week 4 moves the project from architecture definition into active system construction. The team should establish the technical and visual foundation that later pages, content models, and API integrations will build on.

Due to Week 3 shortfalls, this week carries a dual burden: completing the documentation deliverables that were missed last week, and continuing forward with implementation. The documentation gaps must be resolved **before** further implementation proceeds so that the team is not building without a validated plan.

The week focuses on:

- completing Week 3 documentation carry-overs;
- the remaining frontend components not yet built;
- route structure implementation;
- WordPress backend structure;
- REST API preparation;
- shared implementation standards;
- accessibility-aligned frontend behavior;
- multilingual and locale-aware structure.

1.3 2. Week 3 Carry-Over Summary

The following deliverables were required in Week 3 and were not produced. They become Week 4 priority-zero tasks — nothing that depends on them should proceed until they exist.

Deliverable	Owner	Blocks
Design system draft document	Role 4	Role 4 responsive/accessibility work; Role 2 validation
Typography, spacing, and color rules document	Role 4	Role 4 atom review; Role 2 validation
Core wireframes	Role 4	Route planning; page template work
React route strategy document	Role 5	Route implementation in Week 4
Component planning document	Role 5	Role 2 component review; Role 1 validation
Page template planning draft	Role 5	HomePageTemplate, InnerPageTemplate implementation
WordPress content structure notes	Role 6	CPT configuration; ACF field setup
REST API exposure checklist	Role 6	Role 5 API helper implementation
Static vs dynamic content validation notes	Role 6	Week 5 content integration
UX validation notes (W3 components)	Role 2	All further implementation
Frontend strategy validation document	Role 1	Cross-role consistency check

1.4 3. Combined Week 4 Output Checklist

Status reflects actual state entering Week 4.

- ☒ Functional frontend project scaffold — built in Week 3 (Vite + React + TypeScript)
 - ☐ Locale-aware routing structure — not started
 - ☒ Shared component architecture — atoms, molecules, organisms, templates all built in Week 3
 - [~] Design token system implementation — tokens exist in `tailwind.config.js`; specification document missing
 - ☒ Initial reusable UI library — Button, Typography, Card, SectionHeader, Navbar, Footer, HeroSection, AccordionSection, TabsSection, PageLayout, ContentGrid built
 - ☐ WordPress CPT and taxonomy structure — plugins installed, CPTs not yet created
 - ☐ Structured field configuration — not started
 - ☐ REST API preparation — stub only (`getPosts`, `getPost`)
 - ☒ Shared layout templates — PageLayout and ContentGrid built
 - ☐ Accessibility-aligned implementation standards — not documented
-

1.5 4. Implementation Sequencing Rule

The implementation order must remain system-first, not page-first.

1. **Design tokens** before visual components.
2. **Atoms** before molecules.
3. **Molecules** before organisms.
4. **Templates** before full pages.
5. **API helpers** before page-level data usage.
6. **Content model** before final frontend integration.

Note: This rule was violated in Week 3. Role 5 built organisms and templates before the design system was documented and before a route strategy existed. The built components are correct and are kept, but the missing planning documents must be produced this week before further implementation proceeds.

1.6 5. Vite vs Next.js — Open Decision

The original architecture document (`03_Frontend_Architecture.md`) specifies **Next.js 14**. The actual project uses **Vite + React Router**. These are not compatible — migrating later would be a significant rewrite.

Role 1 must resolve this with the team before Week 4 route work begins. Until a decision is recorded, Role 5 should continue on Vite. The frontend scaffold tasks below reflect the Vite reality.

1.7 6. Role 1 — Project Manager + Frontend Lead

1.7.1 Phase 3 Responsibility

System coordination, carry-over resolution, and implementation oversight.

1.7.2 Week 3 Carry-Overs

- [W3] [] Produce standalone **frontend strategy validation document** confirming component architecture aligns with sitemap, user flows, and navigation model.
- [W3] [] Produce standalone **reuse and naming standards document** (currently distributed across sync guide and project rules — needs consolidation).
- [W3] [] Resolve the **Vite vs Next.js** discrepancy — document the decision and update `03_Frontend_Architecture.md` accordingly.

1.7.3 System Coordination Tasks

- ☐ Validate consistency between frontend routes and WordPress structure — blocked until Role 5 produces a route strategy and Role 6 produces backend assumptions.
- ☐ Validate consistency between component architecture and user flows — blocked until Role 2 produces UX validation notes.
- ☐ Supervise implementation priorities for reusable systems.
- ☐ Coordinate communication between frontend and backend contributors.

1.7.4 Implementation Standards Tasks

- ☐ Ensure architecture remains aligned with accessibility goals.
- ☐ Ensure architecture remains aligned with multilingual routing.
- ☐ Ensure architecture remains aligned with REST API structure.
- ☐ Ensure architecture remains aligned with long-term maintainability.

1.7.5 Sequencing Tasks

- ☒ Validate implementation order: atoms before molecules — confirmed, enforced via component folder structure.
- ☒ Validate implementation order: molecules before organisms — confirmed.
- ☒ Validate implementation order: templates before pages — confirmed.
- ☐ Prevent premature page-specific implementation — no real pages started yet; ensure this holds for Week 4.

1.7.6 Deliverables

- ☐ Week 4 implementation supervision notes.
- ☐ Frontend consistency validation notes.
- ☐ Shared implementation standards document.
- ☐ Team coordination updates.

- [W3] [] Frontend strategy validation document.
 - [W3] [] Reuse and naming standards document.
-

1.8 7. Role 2 — UX Researcher + Frontend Contributor

1.8.1 Phase 3 Responsibility

UX validation of Week 3 output and ongoing implementation validation.

Priority: Role 2 produced nothing in Week 3. All Week 3 deliverables are carry-overs. The component library and WordPress plugins were set up without any UX validation pass. This must happen at the start of Week 4 before further components or pages are built.

1.8.2 Week 3 Carry-Overs — Validate Existing Components

These tasks apply to the components already built in Week 3:

- [W3] [] Review **Navbar** against the sitemap — check that the 5 nav links match the agreed top-level routes (**About**, **Erasmus**, **International Students**, **Partnerships**, **Contact**).
- [W3] [] Review **Footer** link structure against the content model — verify section groupings reflect intended information architecture.
- [W3] [] Review **HeroSection** against audience-first navigation requirements — does it surface the right entry points for each user type?
- [W3] [] Review **Card** and **ContentGrid** against user journey flows — are listing patterns appropriate for how users browse programmes, calls, and resources?
- [W3] [] Review **AccordionSection** and **TabsSection** — are these appropriate patterns for the content types they will hold?
- [W3] [] Validate the overall component set against the sitemap and user flows — identify any components that are missing or any that do not fit the agreed structure.
- [W3] [] Produce **UX validation notes** covering the above reviews.
- [W3] [] Produce **route hierarchy corrections** if the sitemap and current component assumptions do not align.
- [W3] [] Produce **navigation consistency feedback**.
- [W3] [] Produce **page-template alignment feedback**.
- [W3] [] Produce **WordPress structure and user-flow feedback** for Role 6.

1.8.3 Week 4 UX Validation Tasks

- ☐ Validate implemented layouts against planned user journeys as new pages are built.
- ☐ Ensure homepage entry points remain visible and audience-oriented.
- ☐ Verify route clarity as Role 5 adds routes.
- ☐ Verify CTA visibility in new templates.
- ☐ Verify content grouping in listing and detail pages.
- ☐ Verify responsive usability of all organisms.

1.8.4 Component Review Tasks

- ☐ Review reusable components for readability.
- ☐ Review reusable components for accessibility.
- ☐ Review reusable components for mobile usability.
- ☐ Review reusable components for consistency with user flows.
- ☐ Ensure document-heavy sections become workflow-oriented layouts.

1.8.5 Deliverables

- [W3] [] UX validation notes (covering Week 3 component output).
 - [W3] [] Route hierarchy corrections.
 - [W3] [] Navigation consistency feedback.
 - [W3] [] Page-template alignment feedback.
 - [W3] [] WordPress structure and user-flow feedback.
 - ☐ Mobile usability feedback (Week 4 components).
 - ☐ CTA and workflow alignment feedback.
-

1.9 8. Role 4 — UI Designer + Frontend Implementer

1.9.1 Phase 3 Responsibility

Design system documentation and remaining atom implementation.

Context entering Week 4: Color tokens (uvt-blue: #005BBB, uvt-navy: #002147, uvt-gold: #F2B705, uvt-gray: #F5F5F5) and spacing tokens (section: 6rem, container: 1280px) exist in `tailwind.config.js`. Heading weight and letter-spacing are set in `globals.css`. No design system document exists. No wireframes were produced. Three atoms (Divider, Icon, LanguageSwitcher) have not been built.

1.9.2 Week 3 Carry-Overs — Documentation First

These must be produced before any further visual implementation:

- [W3] [] Produce **design system draft document** — formalise the visual language. Reference the existing tokens in `tailwind.config.js` and `globals.css` and document the rationale, usage rules, and any gaps.
- [W3] [] Produce **typography, spacing, and color rules document** — specify: heading scale with pixel values, paragraph line-height, link styles, button text sizing, section padding values, card spacing, grid gap rules, mobile overrides, color usage rules per context.
- [W3] [] Produce **core wireframes** for: homepage, internal section page, listing page (programmes/calls/resources), detail page, contact page.

1.9.3 Design Token Tasks

- ☒ Color tokens — defined in `tailwind.config.js` (`uvt-blue`, `uvt-navy`, `uvt-gold`, `uvt-gray`).
- [~] Typography tokens — heading weight/tracking in `globals.css`; no type scale token (`h1–h3` sizes are hardcoded in `Typography.tsx` using Tailwind utility classes `text-5xl`, `text-4xl`, `text-2xl`).
- [~] Spacing tokens — `section: 6rem` and `container: 1280px` in `tailwind.config.js`; card and grid spacing not tokenised.
- ☐ Implement responsive breakpoint rules — document which breakpoints apply to which layout patterns.

1.9.4 UI Atom Tasks

- ☒ Button atom — 3 variants, 3 sizes, focus ring, disabled state.
- ☒ Typography atom — 5 variants (`h1–h3`, `body`, `caption`), correct semantic element per variant.
- ☐ Build Divider atom.
- ☐ Build Icon atom.
- ☐ Build LanguageSwitcher atom.

1.9.5 Responsive Visual Behavior Tasks

- ☐ Test desktop visual behavior.
- ☐ Test tablet visual behavior.
- ☐ Test mobile visual behavior.
- ☐ Check spacing consistency across breakpoints.
- ☐ Check layout readability across breakpoints.

1.9.6 Accessibility-Friendly Visual Pattern Tasks

- ☐ Implement visible focus states — Button already has `focus:ring-2`; check all interactive elements.
- ☐ Check readable contrast — `uvt-blue #005BBB` on white: verify WCAG AA ratio.
- ☐ Check hover states — Navbar and Footer links have hover; verify all interactive elements.
- ☐ Check active states.
- ☐ Align all visual implementation with the design system document once produced.

1.9.7 Deliverables

- [~] Tailwind design token implementation — partial; needs typography and spacing completion.
- ☒ Core UI atoms — Button and Typography built; Divider, Icon, LanguageSwitcher still needed.
- ☐ Responsive styling system.
- ☐ Accessibility-compliant visual patterns.

- [W3] [] Design system draft document.
- [W3] [] Typography, spacing, and color rules document.
- [W3] [] Core wireframes.

1.10 9. Role 5 — React Developer

1.10.1 Phase 3 Responsibility

Route implementation, remaining components, and W3 planning documentation.

Context entering Week 4: The component library is substantially complete. The project uses **Vite + React Router**, not Next.js (see section 5). `App.tsx` has two routes: `/` (UnderConstruction) and `/test` (TestPage). No real content routes exist. The API layer is a stub with two endpoints.

1.10.2 Week 3 Carry-Overs — Documentation First

- [W3] [] Produce **React route strategy document** — map every route in `01_Sitemap.md` to a React route path, specify which are static vs dynamic, which require URL params, and which share a template.
- [W3] [] Produce **component planning document** — document which components are global, which are page-specific, naming conventions, and how the component families relate to each other.
- [W3] [] Produce **page template planning draft** — specify which template each page uses, what organisms it composes, and what data it requires.

1.10.3 Frontend Scaffold Tasks

- ☒ Frontend project initialised — Vite + React + TypeScript.
- ☒ TypeScript configured.
- ☒ Tailwind CSS configured.
- ☐ Configure locale routing — not yet implemented; required by architecture.
- ☒ Frontend folder structure created.
- ☐ Resolve Vite vs Next.js with Role 1 before building locale routing.

1.10.4 Atomic Component Implementation Tasks

- ☒ Atoms implemented — Button, Typography.
- ☒ Molecules implemented — Card, SectionHeader.
- ☒ Shared layout shells implemented — PageLayout, ContentGrid, Container.
- ☐ Implement routing structure — only `/` and `/test` exist; full route tree needed.

1.10.5 Reusable Organism Tasks

- ☒ Navbar built.

- ☒ Footer built.
- ☒ SectionHeader built (molecule level).
- ☒ AccordionSection built.
- ☐ Build DocumentDownloadList — not yet built; required for resource and call pages.

1.10.6 Page Template Tasks

- ☒ PageLayout prepared.
- ☐ Prepare InnerPageTemplate — needed for all section pages.
- ☐ Prepare HomePageTemplate — needed for the homepage.

1.10.7 Route Implementation Tasks

These are new for Week 4 — not in the original plan but required now that the scaffold exists:

- ☐ Implement homepage route (/).
- ☐ Implement top-level section routes (/erasmus, /international-students, /programmes, /partnerships, /calls, /stories, /resources, /contact).
- ☐ Implement nested Erasmus routes (/erasmus/incoming-students, /erasmus/outgoing-students, /erasmus/incoming-staff, /erasmus/outgoing-staff).
- ☐ Implement fallback / 404 route.
- ☐ Connect routes to page files — one file per route in pages/.

1.10.8 API Layer Tasks

- ☐ Add API helper for `programmes` CPT.
- ☐ Add API helper for `calls` CPT.
- ☐ Add API helper for `stories` CPT.
- ☐ Add API helper for `resources` CPT.
- ☐ Expand `getPages()` and `getPosts()` with typed responses.

API helpers should only be added once Role 6 has confirmed the CPT slugs and REST API endpoints via the REST API exposure checklist.

1.10.9 Architecture Separation Rules

- ☒ Presentation separate from fetching — components do not call the API directly.
- ☒ Fetching separate from routing — `api/wordpress.js` is the single fetch layer.
- ☒ Routing separate from templates — `App.tsx` handles routing only.
- ☒ No direct API fetching inside presentational components — confirmed.

1.10.10 Deliverables

- ☒ Frontend project scaffold.
- ☒ Shared component implementation — substantially complete.
- ☒ Layout shell implementation — PageLayout, ContentGrid.
- ☐ Locale-aware routing foundation.

- [W3] [] React route strategy document.
- [W3] [] Component planning document.
- [W3] [] Page template planning draft.

1.11 10. Role 6 — WordPress Developer

1.11.1 Phase 3 Responsibility

Backend documentation carry-overs, CMS structure, and REST API preparation.

Context entering Week 4: ACF and Custom Post Type UI are installed and tracked in the repo. Neither plugin is activated. No CPTs, taxonomies, or field groups have been configured.

1.11.2 Week 3 Carry-Overs — Documentation First

These documents are required before CPT and field configuration begins. Without them, there is no agreed contract for Role 5 to build API helpers against.

- [W3] [] Produce **WordPress content structure notes** — document which content lives in Pages, which in Posts, which in each CPT, and how taxonomies relate to each.
- [W3] [] Produce **REST API exposure checklist** — list every endpoint that the frontend will consume, the expected fields in each response, and whether ACF fields need explicit REST exposure.
- [W3] [] Produce **static vs dynamic content validation notes** — confirm which templates require static WP Pages and which require dynamic CPT queries.
- [W3] [] Fix **folder structure in role fork** — WordPress files are at the repo root in `roles/role6/ro.uvt.ri/`; they must be inside `wordpress/ro.uvt.ri/` to match main.

1.11.3 CMS Structure Tasks

- ☐ Activate ACF and CPTUI plugins in WordPress admin.
- ☐ Configure Pages — create the evergreen page structure matching the sitemap.
- ☐ Configure Posts — confirm post type is used for news/announcements only.
- ☐ Configure Custom Post Types — see CPT tasks below.
- ☐ Configure Taxonomies — see taxonomy tasks below.

1.11.4 Initial CPT Tasks

- ☐ Create `calls` CPT — REST API enabled, `show_in_rest`: true.
- ☐ Create `stories` CPT — REST API enabled.
- ☐ Create `resources` CPT — REST API enabled.
- ☐ Create `programmes` CPT — REST API enabled.
- ☐ Decide whether `people` CPT is necessary — confirm with Role 1.
- ☐ Create `people` CPT if approved.

1.11.5 Taxonomy Tasks

- ☐ Create **audience** taxonomy — attach to all CPTs.
- ☐ Create **programme-family** taxonomy — attach to **programmes**.
- ☐ Create **content-topic** taxonomy — attach to **resources** and **stories**.
- ☐ Create **academic-year** taxonomy — attach to **calls** and **resources**.

1.11.6 Structured Field Tasks

- ☐ Create ACF field group for **calls** — deadline (date), eligibility (textarea), application steps (repeater), documents (repeater: label + file URL).
- ☐ Create ACF field group for **programmes** — duration, language, partner institution, application deadline.
- ☐ Create ACF field group for **resources** — file URL, file type, audience notes.
- ☐ Create ACF field group for **stories** — author, date, pull quote.
- ☐ Prepare CTA group fields (global — used across page types).
- ☐ Prepare process step fields.
- ☐ Prepare document repeater fields.
- ☐ Prepare FAQ section fields.
- ☐ Prepare contact card fields.

1.11.7 REST API Compatibility Tasks

- ☐ Verify each CPT endpoint returns expected fields: `wp-json/wp/v2/{cpt-slug}`.
- ☐ Confirm ACF fields appear in REST responses (enable per field group if not).
- ☐ Prepare locale handling — confirm language/locale field strategy.
- ☐ Prepare taxonomy exposure — verify taxonomy terms appear in CPT REST responses.
- ☐ Prepare structured response consistency — agree field naming with Role 5.
- ☐ Remove legacy or duplicate structural content where possible.
- ☐ Create at least one test entry per CPT for Role 5 to validate against.

1.11.8 Deliverables

- ☐ Initial WordPress structure.
 - ☐ CPT configuration.
 - ☐ Taxonomy configuration.
 - ☐ Structured field setup.
 - ☐ REST API exposure preparation.
 - [W3] [] WordPress content structure notes.
 - [W3] [] REST API exposure checklist.
 - [W3] [] Static vs dynamic content validation notes.
-

1.12 11. Suggested File Outputs

Files that should be created during Week 4. Items marked [W3] were also required last week.

- ☐ `frontend-scaffold-notes.md` — Vite configuration, folder structure, decisions made.
- ☐ `locale-routing-structure.md` — locale routing plan once Vite vs Next.js is resolved.
- ☐ `component-architecture-week-4.md` — component planning document (Role 5).
 - [W3] [] `design-system-draft.md` — full design system specification (Role 4).
 - [W3] [] `typography-spacing-color-rules.md` — standalone rules reference (Role 4).
 - [W3] [] `core-wireframes-notes.md` — wireframe descriptions or links (Role 4).
 - [W3] [] `react-route-strategy.md` — route map from sitemap to React paths (Role 5).
 - [W3] [] `page-template-planning.md` — which template each page uses (Role 5).
 - [W3] [] `wordpress-content-structure-notes.md` — content type assignments (Role 6).
 - [W3] [] `rest-api-exposure-checklist.md` — every endpoint and expected fields (Role 6).
 - [W3] [] `static-vs-dynamic-content.md` — content classification (Role 6).
- ☐ `design-token-implementation.md` — token documentation (Role 4).
- ☐ `ui-atoms-checklist.md` — status of every atom (Role 4).
- ☐ `wordpress-cpt-taxonomy-setup.md` — CPT and taxonomy configuration record (Role 6).
- ☐ `structured-fields-setup.md` — ACF field group record (Role 6).
- ☐ `rest-api-preparation.md` — endpoint verification results (Role 6).
- ☐ `layout-template-implementation.md` — `InnerPageTemplate` and `HomePageTemplate` spec (Role 5).
- ☐ `accessibility-implementation-standards.md` — WCAG targets, focus rules, contrast ratios (Role 4 + Role 1).
- [W3] [] `ux-validation-week3.md` — Role 2 review of existing components.

1.13 12. Definition of Done

Week 4 is complete when:

- ☒ Frontend app runs locally.
- ☒ Tailwind and TypeScript are configured.
- ☐ Locale-aware routing foundation exists.
- ☒ Shared layout shells exist — `PageLayout`, `ContentGrid`.
- ☒ Core atoms and first organisms exist.
- ☐ Remaining atoms built — `Divider`, `Icon`, `LanguageSwitcher`.
- ☐ `DocumentDownloadList` organism built.
- ☐ `InnerPageTemplate` and `HomePageTemplate` prepared.
- ☐ Top-level route structure implemented in `App.tsx`.
- ☐ WordPress has initial CPT and taxonomy structure — plugins activated, CPTs registered.
- ☐ Structured fields are prepared — at least one field group per CPT.
- ☐ REST API exposure preparation is documented and verified.
- ☐ At least one test entry per CPT exists for frontend integration.
- ☐ Design system draft document exists.

- ☐ React route strategy document exists.
 - ☐ REST API exposure checklist exists.
 - ☐ UX validation notes for Week 3 components exist.
 - ☐ Implementation rules are clear for Week 5.
-

1.14 13. Week 5 Handoff Notes

At the end of Week 4, the team should be ready to move from foundations into integration. Week 5 should focus on connecting the frontend route structure to real WordPress data, validating REST responses, expanding page templates, and testing the reusable component system against real content.

Week 5 cannot begin content integration until: - Role 6 has registered CPTs with REST API support and confirmed endpoint responses. - Role 5 has a route strategy that maps to the sitemap. - Role 2 has validated the component set against user journeys. - Role 4 has a design system document that covers all component visual patterns.